



Savitribai Phule Pune University
Centre For Distance and Online Education

SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE
CENTRE FOR DISTANCE AND ONLINE EDUCATION

Program	Master of Computer Application	
Course	Programming from First Principles	
Topic Name	Static and Dynamic Typing Part 1	
Topic Code	-	
Unit/Module No. & Name	7	Type discipline
Self-Learning Hours	-	

Topic Details

S.No	Topic	Pg.No
1.0	Introduction	4
2.0	Meaning And Definition	5-7
3.0	Nature Or Type	8-10
4.0	Examples And Case Studies	11-14
5.0	Keywords And Touch Vocabulary	15



Savitribai Phule Pune University
Centre For Distance and Online Education

OBJECTIVE

1. This paper explains static typing and dynamic typing in computer programming using simple and clear English with an undergraduate academic tone.
2. It defines what a type is and explains how programming languages check types.
3. It compares how different typing choices affect safety, speed, and teamwork in software development.
4. The paper also explains common typing variants such as type inference, duck typing, and gradual typing.
5. It uses examples and short case studies to show how typing decisions influence real programs and learning outcomes.
6. The attached audio could not be transcribed here, so the discussion is based on concepts about typing that are widely taught in programming courses.
7. By the end of the paper, readers should be able to define a data type and explain what type checking means.
8. Readers should also be able to explain that the main difference between static and dynamic typing is when type checks happen.
9. Readers should be able to apply these ideas to common tasks like working with numbers, text, lists, and functions.
10. Readers should also be able to identify practical factors that affect which typing approach is best for a project.
11. These factors include the cost of failure, the size of the codebase, and the mix of skills within a development team.

1.0 INTRODUCTION

Programs work by storing data and applying operations to that data. When a programmer writes code, the programmer also makes assumptions about what each value is. A “type” describes the kind of value, such as a number, a string, or a list. Types matter because different operations are valid for different kinds of values. Adding two integers is sensible, but adding an integer to a sentence is usually not. If a program tries an invalid operation, it may stop with an error, or it may produce an incorrect result that is harder to notice.

Programming languages manage these risks in different ways. Some languages check type rules before the program runs. Other languages check type rules while the program is running. The first style is called static typing, and the second style is called dynamic typing. Both styles appear in widely used languages and both styles have strengths. Understanding them helps students read code, choose tools, and design reliable software.

1.1 Why Types Matter

Types help programmers predict how code will behave. A type system can catch mistakes like calling a text method on a number, indexing into a value that is not a list, or passing the wrong kind of argument to a function. When types are clear, a reader can understand code with fewer guesses. Types also support better tools. Editors can give more accurate suggestions and can warn about likely errors. In team settings, types can act like shared agreements about data shapes, which supports maintenance and reduces confusion during updates.

1.2 Academic Context

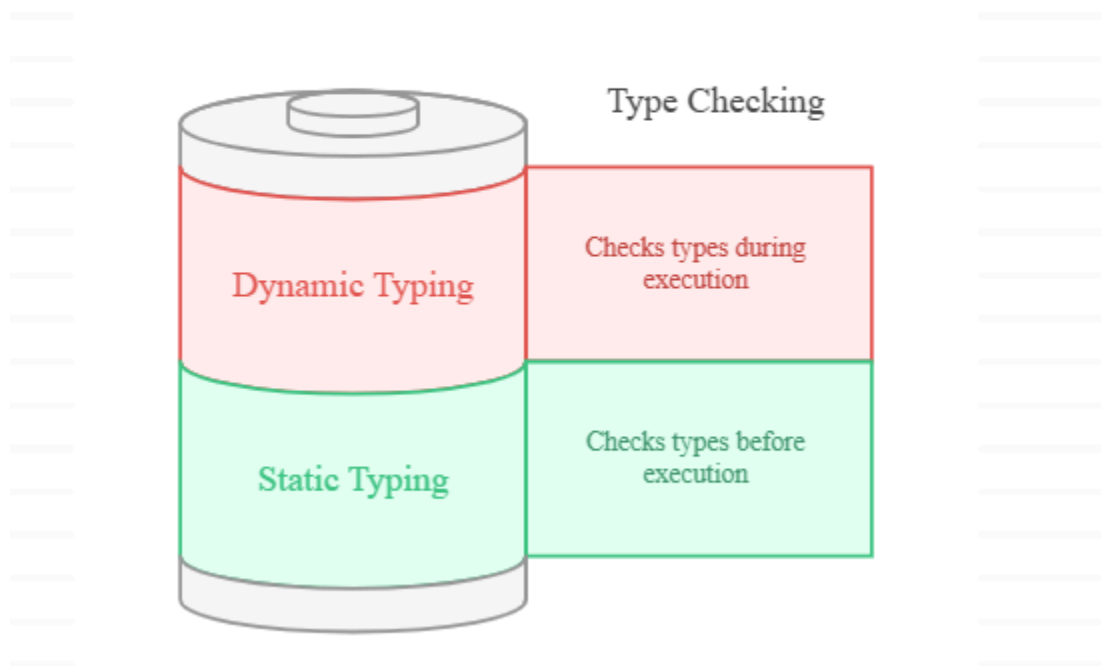
In undergraduate study, typing connects theory and practice. In theory, a type system can be described with rules, and those rules can be used to prove that certain errors cannot occur. In practice, type checks can reduce bugs, improve documentation, and support refactoring. Because these topics affect both correctness and productivity, they are central to modern software engineering. They also connect to debugging, testing, and code review, because all these activities aim to control mistakes and improve confidence in program behavior.

2.0 MEANING AND DEFINITION

A data type is a category of values together with the operations allowed for those values. Common types include integers, floating-point numbers, Booleans, characters, and strings. Languages also provide compound types, such as arrays, lists, maps, and records. Some languages include special types such as null or undefined, which represent missing values. A type system is the set of rules a language uses to assign types to expressions and to decide whether operations are permitted.

Fig. 1

Type checking methods range from early to late detection.



The image shows that type checking can happen at different times in a program. Static typing checks types before execution, while dynamic typing checks types during execution. This illustrates early versus late error detection in programming languages.

2.1 Meaning of Type Checking

Type checking is the process of verifying that values are used in ways that match their types. For example, if a function expects an integer, passing a string is a mismatch. Type

checks can happen before execution, during execution, or in both stages. When a mismatch is found, the language may stop compilation, raise an exception, or otherwise signal an error. The goal is not to eliminate every possible bug, but to remove a common class of errors that come from mixing incompatible kinds of data.

2.1.1 Static Typing

Static typing means type rules are checked before the program runs, often during compilation. The compiler analyzes expressions, variables, and function calls to ensure that each operation is valid for the inferred or declared types. If the checks fail, the compiler reports errors and the program does not run until corrected. This can catch many mistakes early, especially in large codebases where a single change can affect many files.

Static typing can also improve performance because compilers can optimize when types are known. Types also show intent through function signatures. The cost is extra design effort, because the programmer must make types explicit or satisfy the compiler. Sometimes code that would work for one input is rejected if safety cannot be proven.

2.1.2 Dynamic Typing

Dynamic typing means type rules are checked while the program executes. Values carry type information at runtime, and the interpreter or runtime system checks types at the moment an operation occurs. If a program tries an invalid operation, it fails at runtime, usually with an error message that points to a line in the code. Dynamic typing can support fast experimentation, because programmers can write shorter code with fewer declarations and can change how values are used as ideas evolve.

However, dynamic typing can allow type mistakes to remain hidden until a specific path runs with a specific input. This makes testing and careful input checking important. In teaching settings, dynamic typing can reduce early difficulty, but it can also confuse beginners when an error appears far from the original cause, such as when a wrong value travels through many functions before it is used incorrectly. Runtime checks can also add overhead, although many modern runtimes reduce this cost with optimization and caching.

2.2 Meaning of “Type” in Everyday Terms

A helpful analogy is to think of a type as a label on a container. The label tells what is inside and how to handle it. A container labeled “glass” needs care. A value labeled “string” should not be treated like an integer unless it is converted. Types make these

expectations clear, either to the compiler, to the runtime, or to both. This clarity supports reasoning, because the reader can focus on program logic instead of guessing what each variable holds at each moment.

2.3 Definition of Strong and Weak Typing

Static versus dynamic typing is about when checks happen. Strong versus weak typing is about how strictly the language prevents unsafe mixing of types. In strong typing, the language limits silent conversions that could hide mistakes. For example, it may require an explicit conversion from a string to a number. In weak typing, the language may convert values more freely, such as turning text into numbers automatically in some contexts. Because these terms are used differently in different places, it is best to focus on specific rules: what conversions are allowed, and when errors are reported.

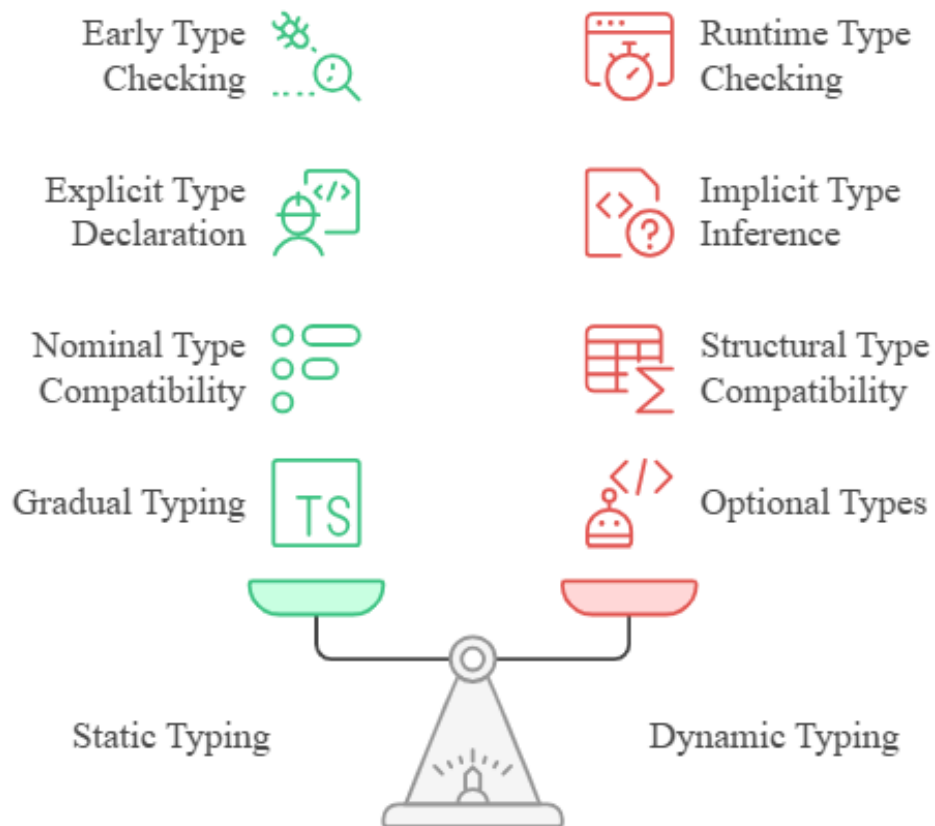


3.0 NATURE OR TYPE

Typing styles include several subtypes. Real languages often blend ideas, and many projects add tools that change how typing is used without changing how the core language executes. This section describes common forms and mixed approaches, and it explains why “static” and “dynamic” are best treated as endpoints on a spectrum rather than as a strict yes-or-no choice.

Fig. 2

Balancing Static and Dynamic Typing



The image compares static and dynamic typing features. Static typing includes early checks, explicit declarations, nominal compatibility, and gradual typing. Dynamic typing includes runtime checks, implicit inference, structural compatibility, and optional types, shown balanced to highlight tradeoffs between both approaches.

3.1 Static Typing: Major Forms

Static typing can be explicit or inferred. In explicit typing, programmers write types in code, such as declaring a variable as an integer or a function as taking a string. In type inference, the compiler deduces types from how values are used. Inference can reduce repetition while preserving early checking. Static systems also support generics, where one function can work with many element types while still remaining safe. For instance, a list type might be written as a list of integers or a list of strings, and the compiler enforces those element rules.

3.1.1 Nominal and Structural Typing

Nominal typing treats two types as compatible when their declared names and relationships match, such as through inheritance. Structural typing treats two types as compatible when they have the same required structure, such as the same fields or methods, even if their names differ. Structural typing can make interfaces flexible, especially when the goal is to accept any object that provides needed behavior. Nominal typing can improve clarity by tying meaning to names, which can be helpful in large systems with many similar shapes.

3.1.2 Gradual Typing and Optional Types

Gradual typing allows static and dynamic typing in the same program. Typed parts are checked early, while untyped parts are checked at runtime. When values cross from untyped to typed code, the system may add runtime checks to keep guarantees. Optional types in tools like TypeScript and Python type hints support a similar pathway: developers can start with flexible code and then add types to the parts that need stronger guarantees. This approach is useful when a team wants to improve safety without rewriting everything at once.

3.2 Dynamic Typing: Major Forms

Dynamic languages vary in how they run. Some interpret code directly, while others use just-in-time compilation to speed repeated work. Many also support reflection, which can be powerful but can make behavior harder to predict without tests and good documentation.

3.2.1 Duck Typing

Duck typing is a dynamic pattern where compatibility is based on behavior. If a value has the needed methods, code will try to use it. This can allow flexible interfaces and quick integration between libraries. In academic terms, duck typing shifts attention from identity to capability: the question becomes “Can this value do what I need?” rather than “Is this value declared as the right class?” The risk is that missing methods may only be discovered during execution, so tests and clear conventions are important.

3.3 Comparing Guarantees and Risks

Static typing aims to prevent many runtime type errors by rejecting unsafe code before execution. The benefit is early feedback and more predictable behavior across many inputs. For example, if a function is declared to take a list of integers, then code that tries to insert a string is rejected. This protects later computations from unexpected values. Dynamic typing accepts more programs up front and then checks operations as they happen. This can make early development faster, but it can move some error discovery into runtime.

The trade-off is when the cost is paid. Static typing pushes effort earlier through design and compiler feedback, aiming to reduce certain runtime failures. Dynamic typing pushes effort later through testing and runtime validation, aiming to keep early iteration fast. Both can be reliable when paired with good engineering practices.

3.4 Effects on Tooling and Teamwork

Static types often improve tooling because the tool can know what members an object has and what arguments a function expects. This supports navigation, autocomplete, and automatic refactoring. For instance, if a field name changes, a static tool can locate many affected places. Dynamic code can still be well tooled, but tools may rely on runtime traces, comments, or optional annotations. When the code uses reflection or dynamic attribute creation, tools may have less certainty.

For teamwork, types can serve as contracts. A module can promise, “This function returns a UserRecord with fields name and id,” and that promise can be checked. Contracts reduce the need for every team member to read internal details. However, contracts can also become rigid if they are too complex or if they change often. A practical approach is to type public boundaries more strictly and allow more flexibility inside modules where experimentation is active.

4.0 EXAMPLES AND CASE STUDIES

The following examples use simple, language-neutral descriptions to highlight the core ideas. The focus is on what happens when values do not match expectations, and how each typing approach helps or hinders discovery of the problem.

4.1 Example: Adding Numbers and Strings

Suppose code tries to add 1 and “2”. In a statically typed setting, the compiler will usually reject the operation unless the programmer converts “2” into a number. This forces a clear decision about parsing and error handling, such as what to do if the string is “two” instead of “2”. In a dynamically typed setting, the runtime will attempt the operation. Some languages raise an error, while others may convert one value into another form. Either way, the issue is detected later, when that line executes.

4.2 Example: Lists With Mixed Items

Consider a list used to store ages. In a statically typed language, the list may be defined as a list of integers. This supports safe operations like summing, sorting numerically, and computing averages. If a string is inserted, the compiler rejects it, and the programmer must decide how to convert the string or where it came from. In a dynamic language, the list can hold integers, strings, and other values. This can be helpful for small scripts, but if later code assumes every item is an integer, a runtime error may appear. A safe practice is to validate data at boundaries, such as when reading files or user input.

4.3 Example: Functions and Return Values

Functions define contracts between caller and callee. In static typing, a function may promise to return a Boolean, and the compiler checks that all return paths match. This prevents many surprises for callers and supports reasoning about control flow, such as when an if-statement expects a Boolean condition. In dynamic typing, functions can return different kinds of values based on conditions. This can be expressive, but it can also place extra burden on the caller to handle many cases and to check types at runtime. A common academic recommendation is to keep return shapes consistent, especially in public interfaces.

4.4 Case Study: A Small Student Script Becomes a Shared Tool

A student may write a short script to process data for a class. Dynamic typing can speed this stage because the student can explore quickly, print intermediate results, and adjust logic without managing many type declarations. Over time, the script may become useful to classmates, and features get added: more inputs, more formats, and more options. Now small type mistakes, such as a missing field name or a list that sometimes contains text, can cause failures for users who do not know the internal details.

At this stage, adding structure becomes valuable. The group might add tests, document input formats, and introduce type hints. If the tool becomes important for a lab or club, they might move core parts to a statically typed language or use a gradual typing tool. The key point is that typing needs can change with scale. Early flexibility can support learning and exploration, while later constraints can support reliability and shared understanding.

4.5 Case Study: Web Development With JavaScript and TypeScript

Web teams often use JavaScript, which is dynamic. For small features, this is fast. For large apps, runtime errors can occur when an object shape changes, such as renaming a field or forgetting to pass a required property. TypeScript adds a static type layer that is checked before execution, and then compiled into JavaScript for deployment. Teams use it to reduce bugs, improve editor support, and enable safer refactoring. TypeScript also illustrates gradual typing: a project can start with few types and tighten rules as it grows.

4.6 Case Study: Data Work in Python With Type Hints

Python is dynamic and common in data work. Its flexibility supports rapid exploration, but pipelines can break when data shapes change. Optional type hints allow developers to describe expected shapes, such as a function returning a list of integers or a dictionary from strings to numbers. A type checker can run in a build step to warn about mismatches, while the runtime still executes dynamically.

In practice, many data tasks start as notebooks or small scripts. At first, the programmer may not know the final structure, so strict types can feel too early. Later, when code turns into a library or a repeated report, errors become expensive. A common strategy is to add type hints at the boundaries of a system. For example, a function that reads a CSV file can promise to return a list of clean records, each record containing named fields with known types. Another function that calls a web API can promise to return a record with

required keys. Then, inside the pipeline, operations can rely on those shapes with less guessing.

4.7 Practical Guidelines for Choosing an Approach

Choosing a typing approach depends on goals. If failures are costly, early checks and clear interfaces are valuable, which often points toward stronger static typing or at least optional typing tools. If the work is exploratory, dynamic typing can reduce friction and speed iteration. In many real projects, a mixed approach is used: dynamic code for experimentation, plus types and tests for the parts that must be stable. The academic takeaway is to treat typing as a design tool, not as a strict ideology. A good design chooses constraints that help the project, and removes constraints that block learning without adding safety.

4.8 Mini Case Study: A Simple Bug in a Message App

Consider a small messaging app that limits messages to 200 characters. A developer reads the limit from a settings file. The file stores values as text, so the limit is read as the string “200”. If the program compares message length (a number) to the limit (text) without conversion, several outcomes are possible. In a strict statically typed language, the compiler rejects the comparison and forces the developer to parse the string into an integer. In a dynamic language, the comparison might raise a runtime error, or it might convert values in a surprising way, depending on the language rules and operators chosen.

1. This mini case study shows why boundaries matter. Input from files, users, and networks should be converted and validated early, so the rest of the program works with consistent data.
2. Across these cases, choose a typing style that matches risk and change. Flexibility helps exploration. Early checks help safety. Tests and validated data boundaries support both in practice.
3. Static typing checks rules before execution and can prevent many runtime type errors, but it may require more upfront planning and explicit interfaces.
4. Dynamic typing checks rules during execution and can speed early development, but it often requires careful testing, validation, and clear conventions.
5. Strong versus weak typing describes how freely a language converts between types; it is not the same as static versus dynamic timing.
6. Gradual typing and optional annotations offer a practical middle path for teams that want flexibility and early feedback together.

5.0 Keyword Touch Vocabulary Meaning

Keyword	Touch Vocabulary	Meaning In Simple Academic English
Type	classification	A label that groups similar values
Type checking	verification	Confirming that operations fit the value type
Static typing	pre-execution checking	Checking types before the program runs
Dynamic typing	runtime checking	Checking types while the program runs
Compiler	translator	A tool that converts and checks code before running
Interpreter	executor	A tool that runs code step by step
Type inference	deduction	The system figures out types from usage
Annotation	note	Written type information in code
Generics	parameterization	Reusable code that works for many types safely
Duck typing	behavior-based use	Using an object because it has needed actions
Gradual typing	hybrid approach	Mixing static and dynamic checks
Coercion	conversion	Automatic change from one type to another